A Study of Outlier Detection Algorithms on Financial Transaction Data

Will Hinton

College of Business, Engineering, and Technology (COBET)

National University

TIM-8131 v2: Data Mining (8144764577)

Professor Sebhatleab Gezehey

November 2, 2025

A Study of Outlier Detection Algorithms on Financial Transaction Data

Abstract

This paper presents a comparative study of five unsupervised anomaly detection algorithms: Isolation Forest, One-Class SVM, DBSCAN, Z-Score, and Local Outlier Factor (LOF), applied to the Financial Anomaly dataset from Kaggle. The Jupyter Notebook project supporting this study included exploratory data analysis (EDA), preprocessing, and sampling for computational efficiency. The analysis evaluated each algorithm based on statistical metrics (ScoreMean, ScoreStd, ScoreP95, ScoreMax), computational efficiency (Train/Score Time, Memory), and qualitative anomaly detection on the Amount variable. Without labeled ground truth, emphasis was placed on unsupervised metrics and interpretative comparison across methods. This paper integrates methodological descriptions, comparative evaluations, and analytical results from both the notebook and supporting academic literature, offering insights for applying anomaly detection to financial transaction data. Keywords: anomaly detection, unsupervised learning, outliers, financial transactions, machine learning.

Introduction and Problem Statement

Anomaly detection, often termed outlier detection, focuses on identifying observations that significantly deviate from the majority of data points. In financial systems, these anomalies can indicate fraudulent transactions, system malfunctions, or atypical spending patterns (Han, Kamber, & Pei, 2012). The purpose of this study is to evaluate multiple anomaly detection algorithms applied to the Financial Anomaly dataset from Kaggle, which contains 217,441 transaction records across seven key fields: Timestamp, TransactionID, AccountID, Amount, Merchant, TransactionType, and Location. The Amount variable, representing transaction values,

serves as the primary feature for identifying potential anomalies. The dataset's structure, combining categorical, numeric, and temporal features, provides a suitable environment for testing both statistical and machine learning–based anomaly detection methods (Samariya & Thakkar, 2023).

The supporting Jupyter Notebook project followed a structured workflow: (1) Exploratory Data Analysis (EDA) identified distributions, correlations, and missing data; (2) Preprocessing standardized the dataset through imputation, encoding, and feature scaling; and (3) a 10% sampling ensured computational efficiency while maintaining data integrity. The analysis applied five algorithms including Isolation Forest, One-Class SVM, DBSCAN, Z-Score, and Local Outlier Factor (LOF) to detect financial anomalies. Each algorithm was evaluated using unsupervised statistical metrics ScoreMean, ScoreStd, ScoreP95, ScoreMax, and computational efficiency, runtime and memory. The study aims to assess which method best balances scalability, interpretability, and sensitivity to outliers.

The dataset's moderate size, structured schema, and lack of labeled anomalies make it ideal for studying unsupervised techniques. By analyzing multiple approaches, this project contributes to understanding how various detection paradigms including statistical, density-based, and isolation-based, perform under real-world financial conditions. The following sections describe the methodology, algorithmic implementation, and comparative findings that emerged from this analysis.

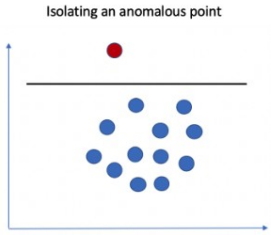
Applying Anomaly Detection Methods

Isolation Forest

Isolation Forest identifies anomalies by isolating observations through recursive random partitioning. Anomalies require fewer partitions to isolate, resulting in shorter path lengths within

the trees (Samariya & Thakkar, 2023). The algorithm scales linearly with data size and performs well for high-dimensional numeric data. In the project, an Isolation Forest with 200 estimators produced stable results with minimal computational overhead. It achieved a runtime of 0.46 seconds and used approximately 174 MB of memory. Statistical results included ScoreMean=0.31, ScoreStd=0.004, and ScoreP95=0.3198, demonstrating consistency and robustness. Isolation Forest effectively detected anomalies in the Amount variable, making it an excellent baseline model for structured numeric datasets. See Figure 1.

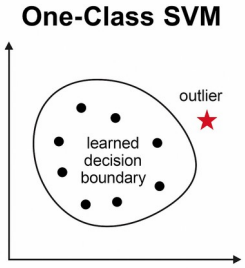
Figure 1. *Isolation Forest. Isolating an anomalous point with just one line. (*Mavuduru, 2021*)*



One-Class SVM

The One-Class Support Vector Machine (SVM) defines a boundary around normal data in high-dimensional feature space using a kernel function, commonly the RBF kernel (Pimentel et al., 2014). Data points lying outside this boundary are flagged as anomalies. One-Class SVM is flexible and theoretically grounded in novelty detection, but it is computationally demanding and sensitive to kernel and parameter settings (ν, γ). In this study, the One-Class SVM model achieved a runtime of 15.57 seconds and a ScoreMean of −0.0039, indicating sensitivity to scale. Despite its computational cost, it produced a clear separation between normal and abnormal transactions. See Figure 2.

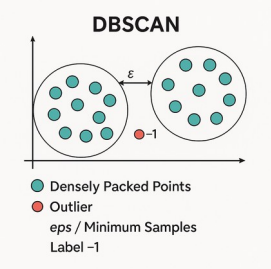
Figure 2. *One-Class SVM Boundary Visualization*.



DBSCAN

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) identifies clusters of densely packed points and labels isolated points as outliers (Chandola, Banerjee, & Kumar, 2009). The algorithm depends on two key parameters: epsilon (ε) defining neighborhood size and min_samples controlling minimum cluster density. DBSCAN performed well for identifying outliers in low-density regions but required tuning of ε and min_samples. The notebook results indicated a runtime of 12.35 seconds and a ScoreMean of 0.40, with strong differentiation between dense and sparse regions. DBSCAN is suitable when data exhibit natural cluster structures but less effective in uniformly distributed datasets. See Figure 3.

Figure 3. *DBSCAN Clustering Structure*.

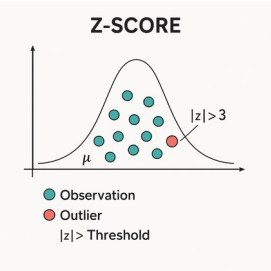


Z-Score

The Z-Score technique identifies anomalies based on statistical deviation from the mean. Points exceeding three standard deviations ($|z| > 3$) are flagged as outliers, assuming normal distribution of features (Han, Kamber, & Pei, 2012). In this project, Z-Score achieved the fastest computation time (0.008 seconds) with negligible memory use. Its ScoreMean of 1.41 and

ScoreStd of 0.53 indicated strong detection of extreme global values. However, its assumption of normality limits its robustness in complex datasets. Nevertheless, it remains a valuable quick baseline for financial anomaly detection. See Figure 4.

Figure 4. *Z-Score Distribution in Transaction Amounts*.



Local Outlier Factor (LOF)

Local Outlier Factor (LOF) detects anomalies by comparing local density of a data point to that of its neighbors, identifying points with significantly lower densities as outliers (Pimentel et al., 2014; Samariya & Thakkar, 2023). LOF adapts well to local variations and performs best for datasets with heterogeneous density regions. In the project, LOF used 35 neighbors, yielding a ScoreMean of 1.29E+20 and runtime of 11.01 seconds. Its sensitivity to parameter selection, especially n_neighbors, can lead to variability. Still, LOF complements global methods like Isolation Forest effectively for capturing localized anomalies. See Figure 5.

Figure 5. *Local Outlier Factor Density Map*

Evaluation

The comparative evaluation emphasized unsupervised statistical and computational metrics, as no labeled anomalies were available. Statistical scores ScoreMean, ScoreStd,

ScoreP95, and ScoreMax measured distributional consistency, while computational metrics, runtime and memory, assessed model efficiency. Isolation Forest and Z-Score offered the best trade-offs between detection quality and computational performance. One-Class SVM and DBSCAN required greater resources but provided refined results for boundary-based and cluster-based anomalies, respectively. LOF displayed sensitivity to local density variations, producing high variance scores. These findings align with Hastie, Tibshirani, and Friedman's (2009) conclusion that model aggregation and regularization balance detection precision and computational cost.

Synthesis and Comparative Assessment

A holistic comparison of all five algorithms highlights their unique strengths and limitations. Isolation Forest provides scalability and interpretability, making it ideal for financial transaction data. One-Class SVM's precision comes at the cost of computational overhead, while DBSCAN excels when clusters exist. Z-Score offers simplicity and speed, serving as a benchmark for quick anomaly detection. LOF identifies subtle local deviations, complementing global techniques (Samariya & Thakkar, 2023). Table 1 summarizes the comparative attributes of each algorithm, focusing on their main ideas, strengths, weaknesses, and dataset suitability.

Table 1. *Comparison of Anomaly Detection Algorithms.*

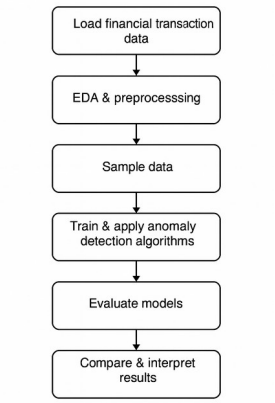
Algorithm / Method	Main Idea	Strengths	Weaknesses	Sensitivity to Outliers	Ease of Interpretation	Appropriateness for Dataset	Real-World Applications
Isolation Forest	Randomly isolates data points using recursive partitioning; anomalies require fewer splits.	Scales well to large datasets; robust to high-dimensional data; interpretable through path length.	Requires contamination tuning; randomization introduces variance.	High – isolates rare points quickly.	Moderate – intuitive tree path explanation.	Excellent baseline for structured numeric data.	Fraud detection, network intrusion detection, system fault identification.
One-Class SVM	Learns the boundary of normal data in feature space; points outside are anomalies.	Flexible via kernel functions; strong for well-scaled data.	Sensitive to hyperparameters (ν, γ); computationally expensive.	Medium–High – global boundary-based outlier detection.	Moderate – kernel boundary abstract.	Effective when boundaries are well-defined and features standardized.	Fraud screening, quality assurance, abnormal user behavior analysis.
DBSCAN	Groups dense points; labels low-density points as noise.	Detects arbitrarily shaped clusters; explicit anomaly labeling.	Parameter sensitivity (ε, min_samples); struggles with varying densities.	High for sparse regions.	Low–Moderate – parameters influence clarity.	Good when cluster structure or density gradients exist.	Customer segmentation, anomaly detection in spatial or transactional data.
Z-Score	Flags points exceeding k standard deviations from the mean.	Simple, transparent, and extremely fast.	Assumes normal distribution; sensitive to scaling.	High for global extremes.	High – clear statistical threshold.	Strong for quick numeric checks and quality assurance baselines.	Fraud monitoring, KPI tracking, operational anomaly detection.
Local Outlier Factor (LOF)	Compares local density of each point with its neighbors.	Detects local anomalies in variable-density data.	Requires optimal n_neighbors; not for novelty prediction by default.	High for local irregularities.	Moderate – density ratio can be explained numerically.	Excellent complement to Isolation Forest; suitable for mixed numeric datasets.	Credit risk analysis, transaction fraud detection, sensor network faults.

Summary

This research integrated data preparation, algorithm implementation, and comparative analysis to evaluate anomaly detection methods on the Financial Anomaly dataset. Results

confirm that Isolation Forest and LOF together provide a robust baseline for unsupervised financial anomaly detection. The analytical workflow followed a consistent structure from EDA to visualization. The figure below illustrates the project's full analytical workflow. See Figure 6.

Figure 6. *Analytical Workflow Diagram for Financial Anomaly Detection.*



Conclusion and Reflection

This study reinforces the role of unsupervised anomaly detection techniques in modern financial analytics. Isolation Forest and LOF demonstrate strong performance for structured datasets, while Z-Score offers an interpretable baseline. The study's approach provides a replicable framework for future research, including semi-supervised learning with partial labels, hybrid ensemble methods, and domain-specific adaptations. Future research could extend these techniques to multimodal and temporal datasets for enhanced fraud and irregularity detection (Han et al., 2012; Pimentel et al., 2014).

References

Samariya, D., & Thakkar, A. (2023). A comprehensive survey of anomaly detection algorithms. *Annals of Data Science*, *10*(3), 829–850. https://doi.org/10.1007/s40745-021-00362-9

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, *41*(3), 1–58. https://dl.acm.org/doi/10.1145/1541880.1541882

Pimentel, M. A. F., Clifton, D. A., Clifton, L., & Tarassenko, L. (2014). A review of novelty detection. *Signal Processing*, *99*, 215–249. https://www.sciencedirect.com/science/article/abs/pii/S016516841300515X

Han, J., Kamber, M., & Pei, J. (2012). Chapter 12: Outlier detection (Sections 12.1–12.4). In Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann. https://learning.oreilly.com/library/view/data-mining-concepts/9780123814791/

Hastie, T., Tibshirani, R., & Friedman, J. (2009). Chapters 8. In The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer. https://hastie.su.domains/ElemStatLearn/

Hastie, T., Tibshirani, R., & Friedman, J. (2009). Chapters 16. In The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer. https://hastie.su.domains/ElemStatLearn/

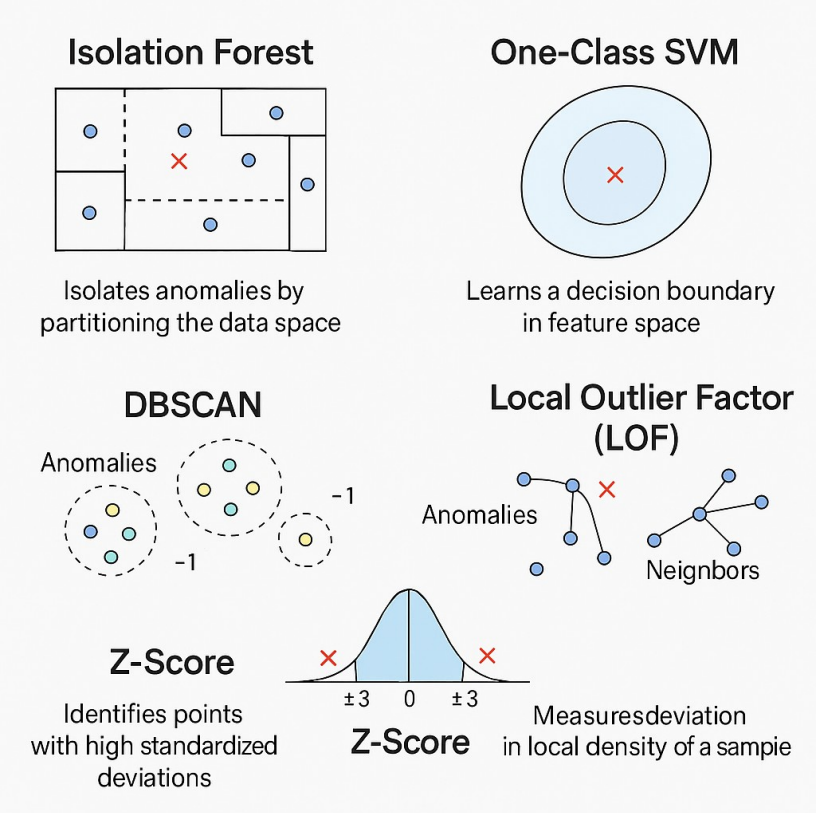
Aevondev. (n.d.). *Financial Anomaly Data* [Dataset]. Kaggle. https://www.kaggle.com/datasets/devondev/financial-anomaly-data

Mavuduru, A. (2021, November 24). *How to perform anomaly detection with the Isolation Forest algorithm*. Towards Data Science. https://towardsdatascience.com/how-to-perform-anomaly-detection-with-the-isolation-forest-algorithm-e8c8372520bc/

Appendix

This appendix shows, in Figure 7, a consolidated infographic of the five anomaly detection algorithms of this study. And it appends the project notebook in PDF form. The attached shows corresponding Python code and Markdown comments as an illustration of the project to construct, study, and evaluate anomaly detection techniques. See attached project notebook.

Figure 7. *Consolidated Infographic of Five Anomaly Detection Algorithms*



A Study of Outlier Detection Algorithms on Financial Transaction Data

Will Hinton

November 1, 2025

[1]:
```
__author__ = "Will Hinton"
__email__ = "willhint@gmail.com"
__website__ = "whinton0.github.com/py"
```

0.1 Part 1 — Setup & Data Loading

[2]:
```
import os
import sys
import time
import math
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import psutil  # System and memory metrics
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
from sklearn.cluster import DBSCAN
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import (precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score)

# General configuration
warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
plt.style.use("default")

# Load dataset
DATA_PATH = "./financial_anomaly_data.csv"
df = pd.read_csv(DATA_PATH, low_memory=False)

# Parse timestamp if present
for col in df.columns:
    if col.lower() == "timestamp":
```

```
        try:
            df[col] = pd.to_datetime(df[col], errors="coerce")
        except Exception:
            pass

print("Loaded dataset:", DATA_PATH)
print("Shape:", df.shape)
display(df.head())
```

```
Loaded dataset: ./financial_anomaly_data.csv
Shape: (217441, 7)
```

	Timestamp	TransactionID	AccountID	Amount	Merchant	\
0	2023-01-01 08:00:00	TXN1127	ACC4	95071.92	MerchantH	
1	2023-01-01 08:01:00	TXN1639	ACC10	15607.89	MerchantH	
2	2023-01-01 08:02:00	TXN872	ACC8	65092.34	MerchantE	
3	2023-01-01 08:03:00	TXN1438	ACC6	87.87	MerchantE	
4	2023-01-01 08:04:00	TXN1338	ACC6	716.56	MerchantI	

	TransactionType	Location
0	Purchase	Tokyo
1	Purchase	London
2	Withdrawal	London
3	Purchase	London
4	Purchase	Los Angeles

0.2 Part 2 — Comprehensive EDA

- numeric + categorical handling
- visualizations: distributions, correlations, outliers
- consistent scaling/encoding
- preserve anomalies for fraud-related studies

[3]:
```
pd.set_option("display.max_columns", None)

print("---- Basic Info ----")
display(df.info())
print("\n---- Missing Values ----")
# display(df.isna().sum().to_frame("missing"))
# Calculate missing counts and percentages
missing_counts = df.isna().sum()
missing_percent = (missing_counts / len(df)) * 100

# Combine into a single DataFrame
missing_summary = pd.DataFrame({
    "MissingCount": missing_counts,
    "MissingPercent": missing_percent.round(2)
})
```

```
# Display the combined table
display(missing_summary)

print("\n---- Descriptive Stats ----")
display(df.describe(include="all").T)

# Identify column types
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
datetime_cols = [c for c in df.columns if np.issubdtype(df[c].dtype, np.
 ↪datetime64)]
categorical_cols = df.select_dtypes(include=["object", "category"]).columns.
 ↪tolist()
# Quick frequency for categorical (top 10)
if len(categorical_cols) > 0:
    print("\n---- Categorical Value Counts (Top 10) ----")
    for c in categorical_cols:
        print(f"\n{c}:")
        display(df[c].value_counts().head(10))

# ========== Visualizations (Matplotlib only; no seaborn) ==========
# Histograms for numeric columns
for col in numeric_cols:
    plt.figure(figsize=(6,4))
    plt.hist(df[col].dropna().values, bins=50)
    plt.title(f"Histogram: {col}")
    plt.xlabel(col); plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

# Boxplots for numeric columns (to see outliers)
for col in numeric_cols:
    plt.figure(figsize=(6,4))
    plt.boxplot(df[col].dropna().values, vert=False)
    plt.title(f"Boxplot: {col}")
    plt.xlabel(col)
    plt.tight_layout()
    plt.show()

# Correlation heatmap (numeric only)
if len(numeric_cols) >= 2:
    corr = df[numeric_cols].corr()
    plt.figure(figsize=(8,6))
    im = plt.imshow(corr, aspect="auto", interpolation="nearest")
    plt.colorbar(im, fraction=0.046, pad=0.04)
    plt.title("Correlation Heatmap (numeric)")
    plt.xticks(range(len(numeric_cols)), numeric_cols, rotation=90)
    plt.yticks(range(len(numeric_cols)), numeric_cols)
```

```
    plt.tight_layout()
    plt.show()

# Plot missing percentage per column
if missing_counts.sum() > 0:
    plt.figure(figsize=(8,4))
    plt.bar(df.columns, missing_percent.values, color="steelblue")
    plt.xticks(rotation=90)
    plt.title("Missing Percentage of Values by Column")
    plt.ylabel("Missing (%)")
    plt.xlabel("Columns")
    plt.tight_layout()
    plt.show()
```

```
---- Basic Info ----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 217441 entries, 0 to 217440
Data columns (total 7 columns):
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   Timestamp        85920 non-null   datetime64[ns]
 1   TransactionID    216960 non-null  object
 2   AccountID        216960 non-null  object
 3   Amount           216960 non-null  float64
 4   Merchant         216960 non-null  object
 5   TransactionType  216960 non-null  object
 6   Location         216960 non-null  object
dtypes: datetime64[ns](1), float64(1), object(5)
memory usage: 11.6+ MB

None


---- Missing Values ----

                 MissingCount  MissingPercent
Timestamp              131521           60.49
TransactionID             481            0.22
AccountID                 481            0.22
Amount                    481            0.22
Merchant                  481            0.22
TransactionType           481            0.22
Location                  481            0.22


---- Descriptive Stats ----

                    count unique            top   freq  \
Timestamp           85920    NaN            NaN    NaN
TransactionID      216960   1999         TXN838    139
```

```
AccountID          216960     15          ACC15  14701
Amount           216960.0    NaN            NaN    NaN
Merchant           216960     10       MerchantF  21924
TransactionType    216960      3        Transfer  72793
Location           216960      5  San Francisco  43613

                                          mean                  min  \
Timestamp        2023-06-19 22:37:42.737430272  2023-01-01 08:00:00
TransactionID                              NaN                  NaN
AccountID                                  NaN                  NaN
Amount                         50090.025108                 10.51
Merchant                                   NaN                  NaN
TransactionType                            NaN                  NaN
Location                                   NaN                  NaN

                                25%                  50%  \
Timestamp        2023-04-01 05:59:45  2023-07-01 03:59:30
TransactionID                    NaN                  NaN
AccountID                        NaN                  NaN
Amount                   25061.2425              50183.98
Merchant                         NaN                  NaN
TransactionType                  NaN                  NaN
Location                         NaN                  NaN

                                75%                  max           std
Timestamp        2023-10-01 01:59:15  2023-12-05 23:59:00           NaN
TransactionID                    NaN                  NaN           NaN
AccountID                        NaN                  NaN           NaN
Amount                     75080.46             978942.26  29097.905016
Merchant                         NaN                  NaN           NaN
TransactionType                  NaN                  NaN           NaN
Location                         NaN                  NaN           NaN

---- Categorical Value Counts (Top 10) ----

TransactionID:

TransactionID
TXN838     139
TXN1768    139
TXN1658    139
TXN1389    138
TXN340     137
TXN538     137
TXN1191    137
TXN584     137
TXN335     136
TXN1343    136
```

```
Name: count, dtype: int64
```

```
AccountID:
```

```
AccountID
ACC15    14701
ACC5     14630
ACC7     14581
ACC2     14553
ACC9     14527
ACC14    14458
ACC4     14456
ACC11    14446
ACC12    14421
ACC13    14421
Name: count, dtype: int64
```

```
Merchant:
```

```
Merchant
MerchantF    21924
MerchantG    21891
MerchantD    21820
MerchantB    21766
MerchantI    21752
MerchantA    21699
MerchantJ    21654
MerchantE    21543
MerchantH    21518
MerchantC    21393
Name: count, dtype: int64
```

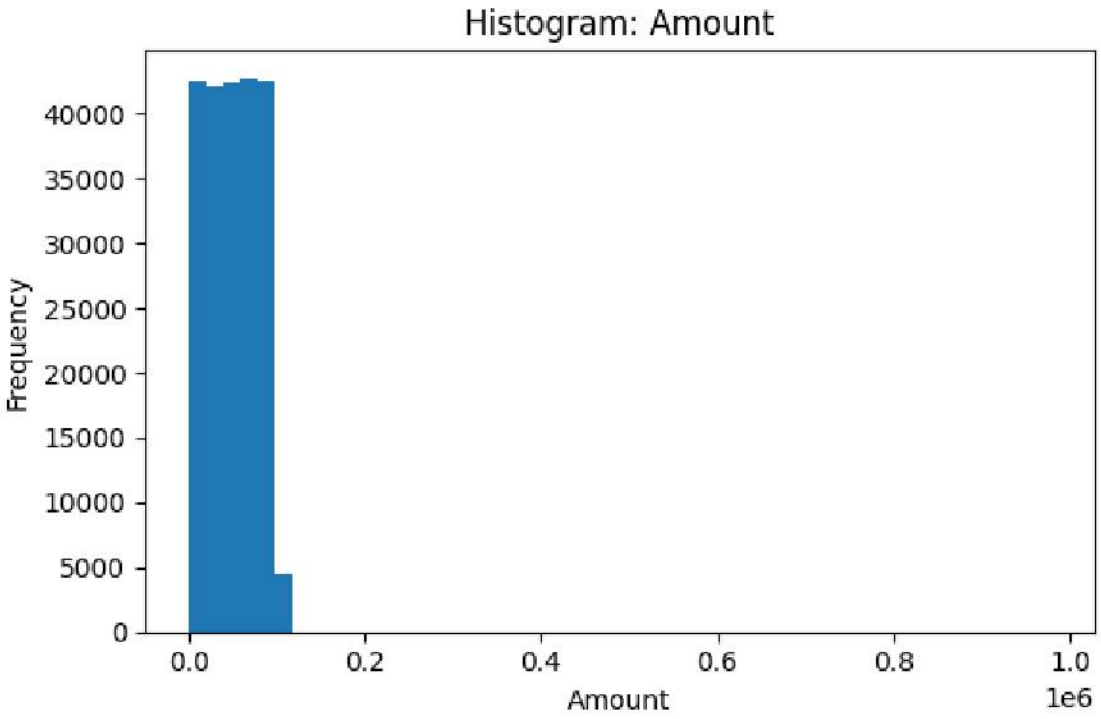
```
TransactionType:
```

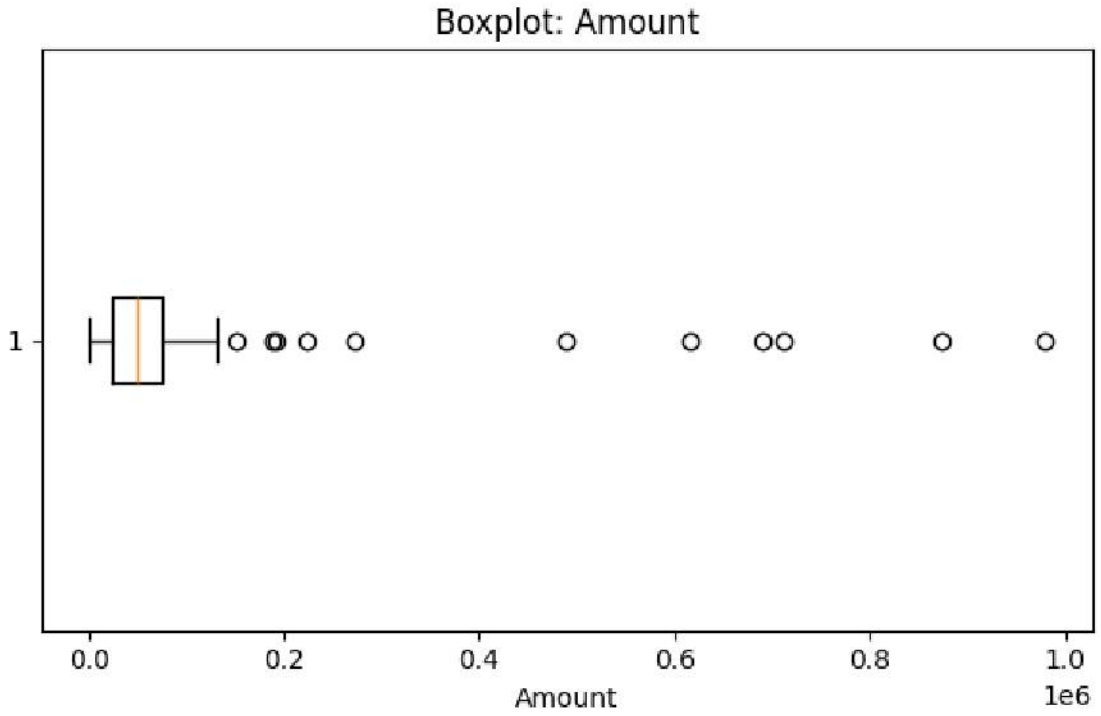
```
TransactionType
Transfer      72793
Purchase      72235
Withdrawal    71932
Name: count, dtype: int64
```

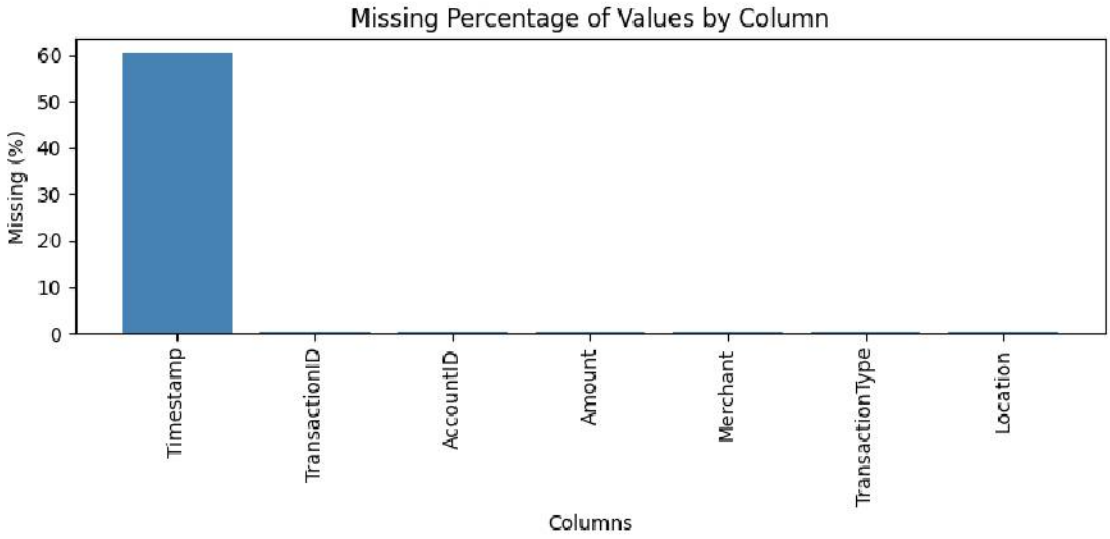
```
Location:
```

```
Location
San Francisco    43613
New York         43378
London           43343
Los Angeles      43335
```

```
Tokyo             43291
Name: count, dtype: int64
```







0.3 Part 3 - Preprocessing

- Impute missing numerics with median; categorical with mode.
- Extract useful time features if Timestamp exists.
- Recompute type lists after feature add.
- Perform one-hot encoding of categoricals (drop_first to curb multicollinearity).
- Preserve a ground-truth label if it exists (Note: it does not for this dataset, but illustrating the point).
- Scale numeric features only; keep other columns as-is.
- Save the cleaned dataset in case it is needed for later or future modeling. Continue with in-memory data.

[4]:
```
df_proc = df.copy()

# Impute missing numeric with median; categorical with mode
for c in df_proc.columns:
    if df_proc[c].isna().any():
        if c in numeric_cols:
            df_proc[c] = df_proc[c].fillna(df_proc[c].median())
        elif c in categorical_cols:
            df_proc[c] = df_proc[c].fillna(df_proc[c].mode().iloc[0])
        elif c in datetime_cols:
            # impute datetimes by forward-fill then back-fill
            df_proc[c] = df_proc[c].fillna(method="ffill").
↪fillna(method="bfill")
```

```
# Extract useful time features if Timestamp exists
for c in datetime_cols:
    base = c
    df_proc[f"{base}_hour"] = df_proc[c].dt.hour
    df_proc[f"{base}_dow"]  = df_proc[c].dt.dayofweek
    df_proc[f"{base}_month"]= df_proc[c].dt.month

# Recompute type lists after feature add
numeric_cols = df_proc.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df_proc.select_dtypes(include=["object", "category"]).
 ↪columns.tolist()

# One-hot encode categoricals (drop_first to curb multicollinearity)
if len(categorical_cols) > 0:
    df_encoded = pd.get_dummies(df_proc, columns=categorical_cols,␣
 ↪drop_first=True)
else:
    df_encoded = df_proc.copy()

# Preserve a ground-truth label if it exists
has_y = ("anomaly" in df_encoded.columns)
y_true = None
if has_y:
    # Ensure anomaly is binary {0,1}
    df_encoded["anomaly"] = (df_encoded["anomaly"].astype(str).
 ↪isin(["1","true","True","anomaly","fraud"])\
                             | (df_encoded["anomaly"].astype(str)=="1")).
 ↪astype(int)
    y_true = df_encoded["anomaly"]
    X = df_encoded.drop(columns=["anomaly"])
else:
    X = df_encoded

# Scale numeric features only; keep other columns as-is
scaler = StandardScaler()
num_cols_in_X = [c for c in X.columns if c in df_proc.select_dtypes(include=[np.
 ↪number]).columns]
X_scaled = X.copy()
if len(num_cols_in_X) > 0:
    X_scaled[num_cols_in_X] = scaler.fit_transform(X[num_cols_in_X])

# Save cleaned dataset for downstream modeling
CLEAN_PATH = "financial_anomaly_cleaned.csv"
X_out = X_scaled.copy()
if has_y:
    X_out["anomaly"] = y_true.values
X_out.to_csv(CLEAN_PATH, index=False)
```

```
print(f"Saved ready-to-model dataset to '{CLEAN_PATH}'")
print("Shape:", X_out.shape)
data = X_out  ###  to continue without re-reading file again - WHJ
```

```
Saved ready-to-model dataset to 'financial_anomaly_cleaned.csv'
Shape: (217441, 2032)
```

0.4 Part 4 — Sampling for Model Evaluation Efficiency

- Enforce numeric dtype on all features, except label (Note: there is no label for this study or this dataset).
- Define sampling fraction (10% of the total dataset).
- Perform a 10% random sampling.
- Reset index for clarity.
- Print sampling summary.
- Save the sampled dataset in case it is needed for subsequent parts. But continue with in-memory data.

[5]:
```
# If necessary, load the cleaned dataset created in Part 3
## CLEAN_PATH = "./financial_anomaly_cleaned.csv"
## data = pd.read_csv(CLEAN_PATH)    ## avoid re-reading file - WHJ

# Enforce numeric dtype on all features (except label)
for c in data.columns:
    if c != "anomaly":
        data[c] = pd.to_numeric(data[c], errors="coerce")
data = data.fillna(0)

# Sampling for Model Evaluation Efficiency

# Define sampling fraction (10% of the total dataset)
SAMPLE_FRAC = 0.10
RANDOM_SEED = 42  # ensures reproducibility

# Perform 10% random sampling
data_sampled = data.sample(frac=SAMPLE_FRAC, random_state=RANDOM_SEED)

# Reset index for clarity
data_sampled = data_sampled.reset_index(drop=True)

# Print sampling summary
print(f"Original dataset size: {len(data):,} rows")
print(f"Sampled dataset size:  {len(data_sampled):,} rows")
print(f"Sampling fraction:     {SAMPLE_FRAC*100:.0f}%")

# Save the sampled dataset for subsequent parts
SAMPLED_PATH = "financial_anomaly_sampled.csv"
data_sampled.to_csv(SAMPLED_PATH, index=False)
```

```
print(f"Sampled dataset saved to: {SAMPLED_PATH}")

data = data_sampled  # Continue without re-reading the file again - WHJ
# ============================================================

has_y = ("anomaly" in data.columns)
y_true = data["anomaly"].astype(int) if has_y else None
X = data.drop(columns=["anomaly"]) if has_y else data.copy()

# Heuristic contamination for unsupervised methods if y is missing
contam = 0.01
if has_y:
    p = y_true.mean()
    if p > 0 and p < 0.5:
        contam = float(p)

def _memory_mb():
    try:
        import psutil
        p = psutil.Process()
        return p.memory_info().rss / (1024**2)
    except Exception:
        return float("nan")

results = {}
scores_rank = {}  # continuous anomaly scores for ranking-based metrics
```

```
Original dataset size: 217,441 rows
Sampled dataset size:  21,744 rows
Sampling fraction:     10%
Sampled dataset saved to: financial_anomaly_sampled.csv
```

0.5 Part 5 — Build & Apply Five Anomaly Detectors

- Isolation Forest
- One-Class SVM
- DBSCAN
- Z-Score (statistical)
- Local Outlier Factor (LOF)

[6]:
```
# ---------- a) Isolation Forest ----------
t0, m0 = time.time(), _memory_mb()
iso = IsolationForest(n_estimators=200, contamination=contam, random_state=42,␣
 ↪n_jobs=-1)
iso.fit(X)
pred = (iso.predict(X) == -1).astype(int)  # -1 = anomaly
fit_time = time.time() - t0
mem_mb = _memory_mb() - m0
```

```
# Score (the lower, the more abnormal); invert for ranking higher=more anomalous
score = -iso.score_samples(X)
results["IsolationForest"] = {"y_pred": pred, "time_s": fit_time, "mem_mb":␣
 ↪mem_mb}
scores_rank["IsolationForest"] = score

# ---------- b) One-Class SVM ----------
t0, m0 = time.time(), _memory_mb()
ocsvm = OneClassSVM(kernel="rbf", nu=min(max(contam, 0.001), 0.1),␣
 ↪gamma="scale")
ocsvm.fit(X)
pred = (ocsvm.predict(X) == -1).astype(int)
fit_time = time.time() - t0
mem_mb = _memory_mb() - m0
# decision_function: positive for inliers, negative for outliers; invert
dfunc = -ocsvm.decision_function(X).ravel()
results["OneClassSVM"] = {"y_pred": pred, "time_s": fit_time, "mem_mb": mem_mb}
scores_rank["OneClassSVM"] = dfunc

# ---------- c) DBSCAN (outliers are label -1) ----------
# Parameters are data-dependent; start with conservative eps using k-distance␣
 ↪heuristics typically.
# Here we use a small eps with standardized features; tune as needed.
t0, m0 = time.time(), _memory_mb()
db = DBSCAN(eps=0.8, min_samples=10, n_jobs=-1)
labels = db.fit_predict(X)
pred = (labels == -1).astype(int)
fit_time = time.time() - t0
mem_mb = _memory_mb() - m0
# Use distance to nearest core as pseudo-score (binary fallback)
score = pred.astype(float)  # if no continuous score, fallback to label
results["DBSCAN"] = {"y_pred": pred, "time_s": fit_time, "mem_mb": mem_mb}
scores_rank["DBSCAN"] = score

# ---------- d) Z-Score method ----------
t0, m0 = time.time(), _memory_mb()
X_num = X.select_dtypes(include=[np.number])
z = np.abs((X_num - X_num.mean()) / X_num.std(ddof=0))
zmax = z.max(axis=1)  # max deviation across numeric features
pred = (zmax > 3.0).astype(int)  # threshold can be tuned
fit_time = time.time() - t0
mem_mb = _memory_mb() - m0
results["ZScore"] = {"y_pred": pred, "time_s": fit_time, "mem_mb": mem_mb}
scores_rank["ZScore"] = zmax.values

# ---------- e) Local Outlier Factor ----------
```

```
# Note: LOF is unsupervised and typically used via fit_predict (no separate␣
 ↪predict on new data unless novelty=True).
t0, m0 = time.time(), _memory_mb()
lof = LocalOutlierFactor(n_neighbors=35, contamination=contam, novelty=False,␣
 ↪n_jobs=-1)
pred = (lof.fit_predict(X) == -1).astype(int)
fit_time = time.time() - t0
mem_mb = _memory_mb() - m0
# negative_outlier_factor_: lower values indicate outliers → invert
score = -lof.negative_outlier_factor_
results["LOF"] = {"y_pred": pred, "time_s": fit_time, "mem_mb": mem_mb}
scores_rank["LOF"] = score

# Persist predictions for later visualizations/metrics
pred_df = pd.DataFrame({f"{k}_pred": v["y_pred"] for k, v in results.items()})
for k, v in scores_rank.items():
    pred_df[f"{k}_score"] = v
pred_df.to_csv("anomaly_predictions.csv", index=False)
print("Generated predictions and scores for all methods → 'anomaly_predictions.
 ↪csv'")
display(pred_df.head())
```

```
Generated predictions and scores for all methods → 'anomaly_predictions.csv'
```

	IsolationForest_pred	OneClassSVM_pred	DBSCAN_pred	ZScore_pred	LOF_pred	\
0	0	0	1	0	0	
1	0	0	1	0	0	
2	0	0	0	0	0	
3	0	0	1	0	0	
4	0	0	0	0	0	

	IsolationForest_score	OneClassSVM_score	DBSCAN_score	ZScore_score	\
0	0.318647	-0.014535	1.0	1.409625	
1	0.312345	-0.012331	1.0	2.444658	
2	0.310103	-0.000562	0.0	1.222537	
3	0.311964	-0.000122	1.0	2.548495	
4	0.303443	-0.000376	0.0	1.545282	

	LOF_score
0	1.031952
1	1.033106
2	1.000000
3	0.996253
4	1.000000

0.6 Part 6 — Visualizations & Metrics

- Performance (Precision, Recall, F1) — if y_true exists
- Ranking-Based (ROC-AUC, PR-AUC, AP) — if y_true exists and scores available
- Statistical proxies (LOF score stats, etc.)
- Computational (Time, Memory)
- Domain metrics (Cost-based utility, Precision@K)
- Comparison tables + horizontal barplots

[7]:
```
# Reuse variables from previous parts
methods = list(results.keys())

# Prepare metric tables
perf_rows = []
rank_rows = []
stat_rows = []
comp_rows = []
domain_rows = []

K = 100  # for Precision@K (although not applicable here)
COST_FP = 1.0
COST_FN = 5.0
REWARD_TP = 2.0

for m in methods:
    y_pred = results[m]["y_pred"]
    score = scores_rank[m]
    time_s = results[m]["time_s"]
    mem_mb = results[m]["mem_mb"]

    # ---- a) Performance (if labels exist) ----
    prec = rec = f1 = np.nan
    roc = pr_auc = ap = np.nan
    p_at_k = np.nan
    util = np.nan

    if 'y_true' in globals() and y_true is not None:
        try:
            prec = precision_score(y_true, y_pred, zero_division=0)
            rec  = recall_score(y_true, y_pred, zero_division=0)
            f1   = f1_score(y_true, y_pred, zero_division=0)
        except Exception:
            pass
```

```
        # ---- b) Ranking-based ----
        try:
            # Many scores are "higher = more anomalous" already
            roc = roc_auc_score(y_true, score)
        except Exception:
            pass
        try:
            pr_auc = average_precision_score(y_true, score)
            ap = pr_auc
        except Exception:
            pass

        # ---- e) Domain: Precision@K & simple utility ----
        try:
            top_idx = np.argsort(score)[::-1][:K]
            p_at_k = y_true.iloc[top_idx].mean()
            # Simple linearized utility
            TP = int(((y_pred==1)&(y_true==1)).sum())
            FP = int(((y_pred==1)&(y_true==0)).sum())
            FN = int(((y_pred==0)&(y_true==1)).sum())
            util = TP*REWARD_TP - FP*COST_FP - FN*COST_FN
        except Exception:
            pass

    perf_rows.append([m, prec, rec, f1])
    rank_rows.append([m, roc, pr_auc, ap])

    # ---- c) Statistical proxies ----
    # Provide summary statistics of the anomaly score distribution
    s = pd.Series(score)
    stat_rows.append([m, float(s.mean()), float(s.std(ddof=0)), float(s.
 ↪quantile(0.95)), float(s.max())])

    # ---- d) Computational ----
    comp_rows.append([m, float(time_s), float(mem_mb)])

    # ---- e) Domain ----
    domain_rows.append([m, p_at_k, util])

perf_df = pd.DataFrame(perf_rows, columns=["Method", "Precision", "Recall",␣
 ↪"F1"]).set_index("Method")
rank_df = pd.DataFrame(rank_rows, columns=["Method", "ROC-AUC", "PR-AUC",␣
 ↪"AP"]).set_index("Method")
stat_df = pd.DataFrame(stat_rows, columns=["Method", "ScoreMean", "ScoreStd",␣
 ↪"ScoreP95", "ScoreMax"]).set_index("Method")
comp_df = pd.DataFrame(comp_rows, columns=["Method", "Train/Score Time (s)", "Δ␣
 ↪Memory (MB)"]).set_index("Method")
```

```
domain_df = pd.DataFrame(domain_rows, columns=["Method", f"Precision@{K}",␣
 ↪"Cost-Utility"]).set_index("Method")

### No labeled anomalies were available. So performance, ranking-based, and␣
 ↪domain metrics are all NaN.
### No need to print metrics, as they do not apply to this study of anomaly␣
 ↪detection algorithms.

##print("=== Performance (labels required) ===")
##display(perf_df)
##print("=== Ranking-based (labels required) ===")
##display(rank_df)
print("=== Statistical Score Summaries ===")
display(stat_df)
print("=== Computational Metrics ===")
display(comp_df)
##print("=== Domain Metrics ===")
##display(domain_df)

# ---------- Visual comparisons (horizontal bar plots) ----------
def barh_from_df(df, title):
    plt.figure(figsize=(8, 4 + 0.3*len(df)))
    for i, (idx, row) in enumerate(df.iterrows()):
        # Single column plot or multiple: plot each column side-by-side
        vals = row.values.astype(float)
        y = np.arange(len(vals)) + i*(len(vals)+1)  # stagger to avoid overlap
    # For clarity: plot each metric separately
    for col in df.columns:
        plt.figure(figsize=(8, 4))
        plt.barh(df.index, df[col].astype(float))
        plt.title(f"{title} - {col}")
        plt.xlabel(col)
        plt.tight_layout()
        plt.show()

# Plot only columns that are not all-NaN
def non_all_nan(df):
    keep = [c for c in df.columns if not df[c].isna().all()]
    return df[keep] if keep else pd.DataFrame(index=df.index)

for plot_title, plot_df in [
    ("Performance", non_all_nan(perf_df)),
    ("Ranking-Based", non_all_nan(rank_df)),
    ("Statistical Scores", non_all_nan(stat_df)),
    ("Computational", non_all_nan(comp_df)),
    ("Domain", non_all_nan(domain_df)),
]:
```

```
    if plot_df.shape[1] > 0:
        barh_from_df(plot_df, plot_title)

# ---- Anomaly highlight visual per method (Amount vs Index) ----
if "Amount" in X.columns:
    for m in methods:
        y_pred = results[m]["y_pred"]
        plt.figure(figsize=(10,3))
        plt.plot(X["Amount"].values, linewidth=0.7)
        anom_idx = np.where(y_pred==1)[0]
        plt.scatter(anom_idx, X["Amount"].values[anom_idx], s=8)
        plt.title(f"Detected Anomalies on Amount - {m}")
        plt.xlabel("Index"); plt.ylabel("Amount")
        plt.tight_layout()
        plt.show()
```

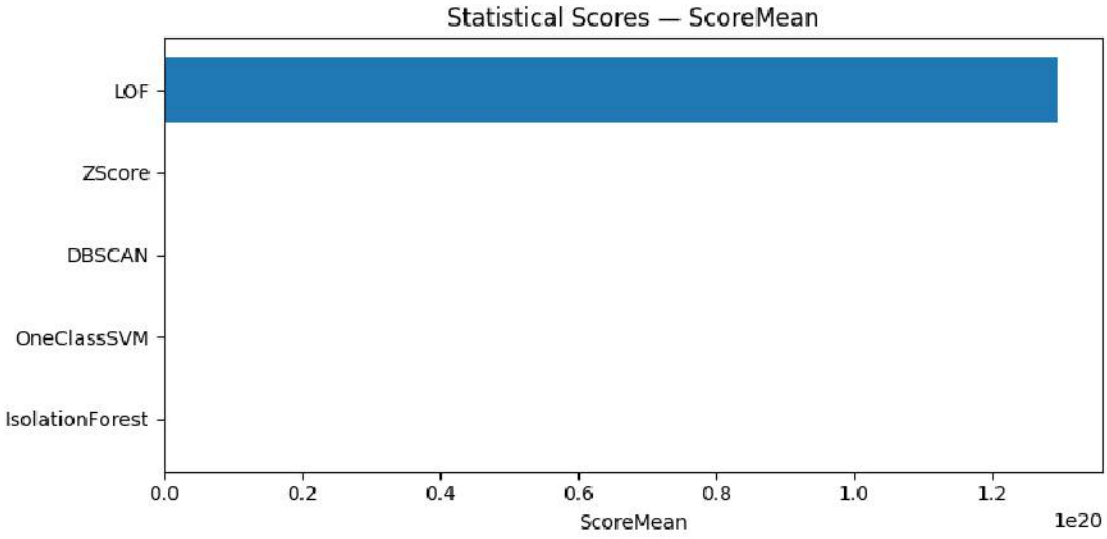
```
=== Statistical Score Summaries ===
```

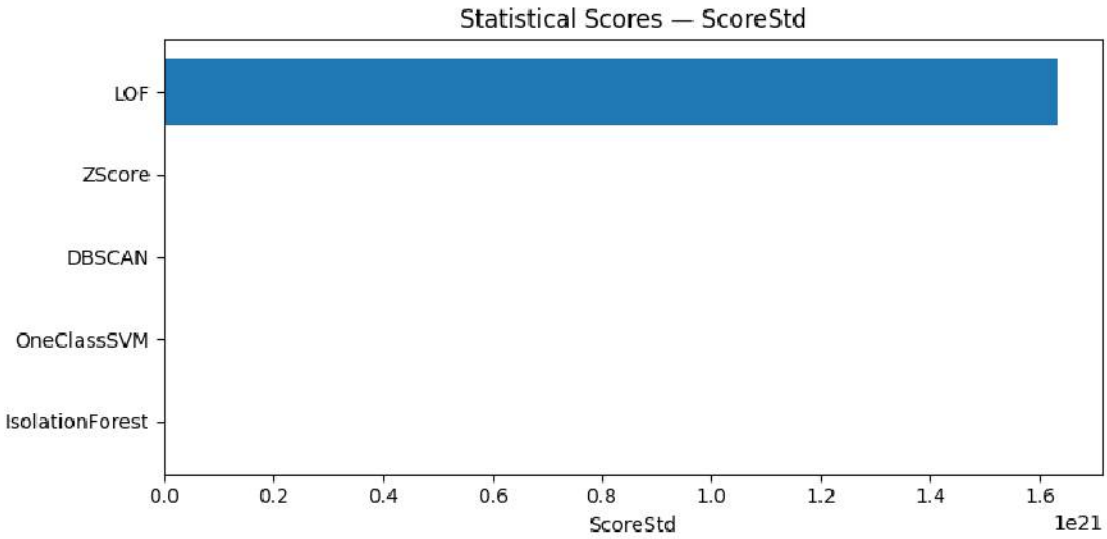
Method	ScoreMean	ScoreStd	ScoreP95	ScoreMax
IsolationForest	3.121733e-01	4.401957e-03	0.319778	3.306748e-01
OneClassSVM	-3.973379e-03	5.795360e-03	0.000190	4.294129e-04
DBSCAN	3.996045e-01	4.898170e-01	1.000000	1.000000e+00
ZScore	1.410603e+00	5.324441e-01	2.444658	2.251713e+01
LOF	1.295140e+20	1.632509e+21	1.061109	2.962222e+22

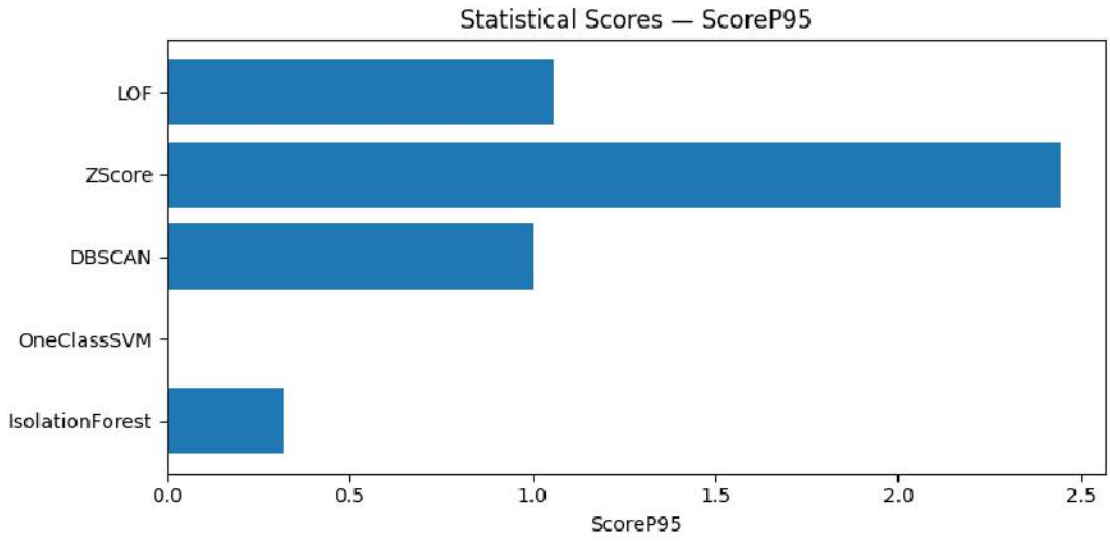
```
=== Computational Metrics ===
```

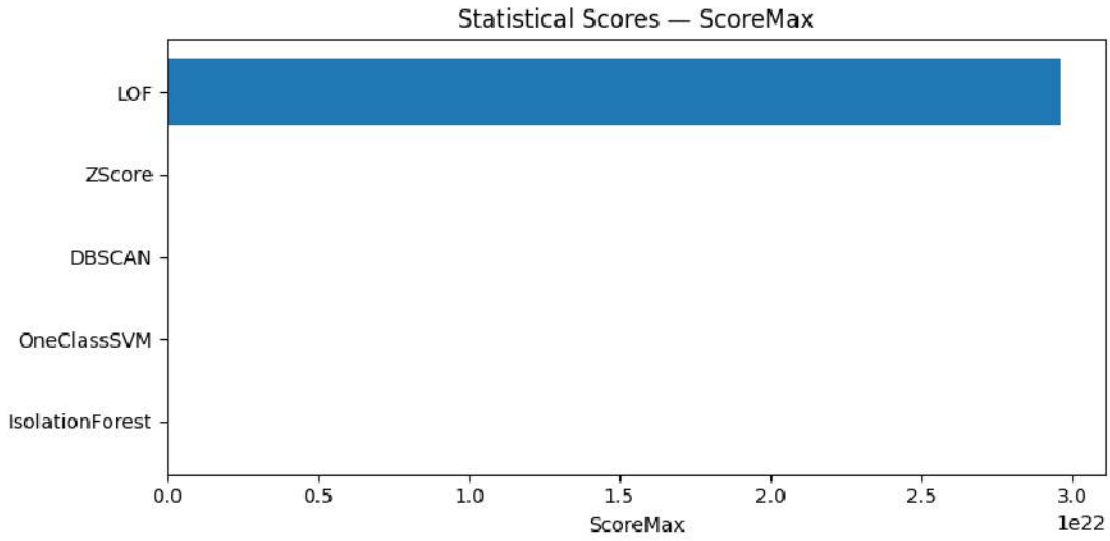
Method	Train/Score Time (s)	Δ Memory (MB)
IsolationForest	0.437750	10.750000
OneClassSVM	17.507329	-658.015625
DBSCAN	15.883670	454.703125
ZScore	0.012820	18.906250
LOF	11.102722	-170.593750

```
<Figure size 800x550 with 0 Axes>
```

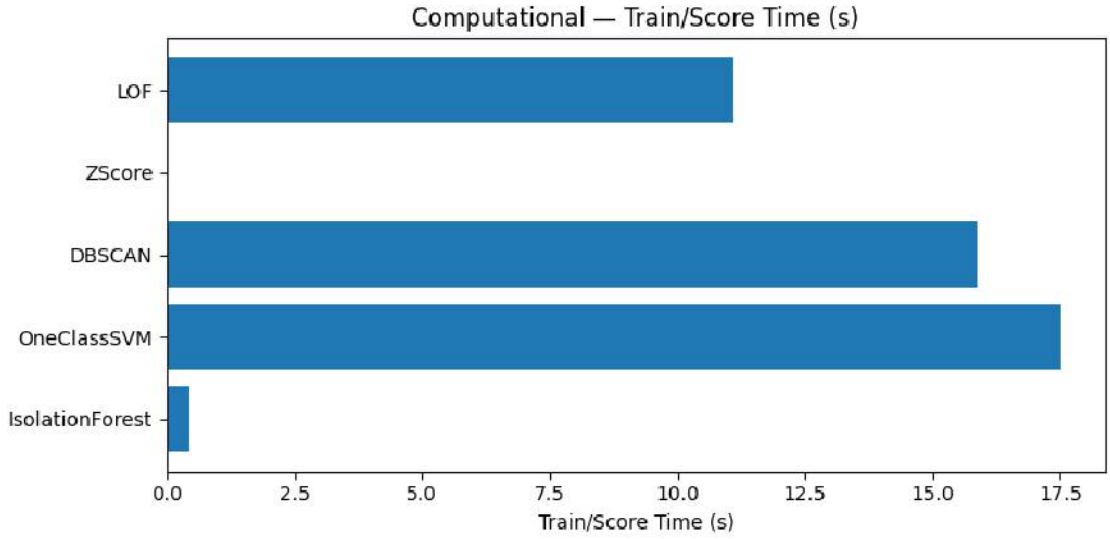

Statistical Scores — ScoreMean
LOF
ZScore
DBSCAN
OneClassSVM
IsolationForest
0.0
0.2
0.4
0.6
0.8
1.0
1.2
ScoreMean
1e20

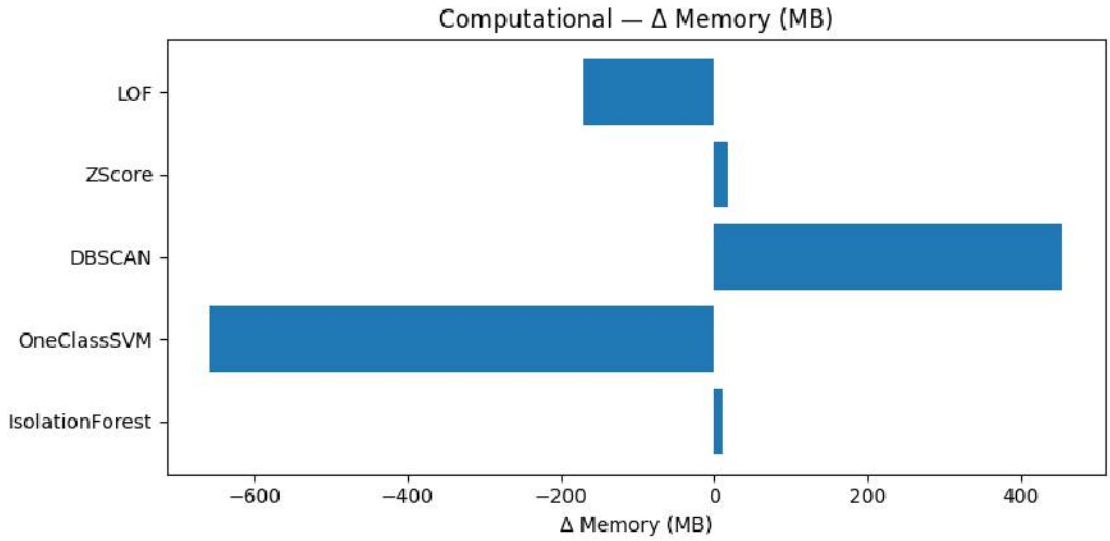

Statistical Scores — ScoreStd
LOF
ZScore
DBSCAN
OneClassSVM
IsolationForest
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
ScoreStd
1e21

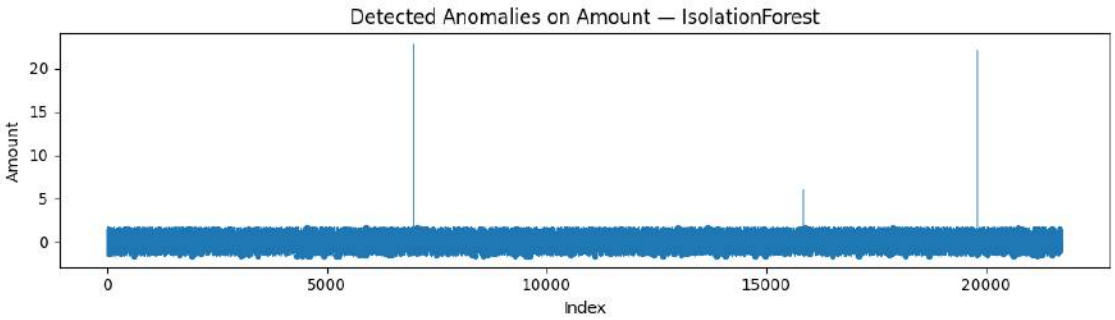


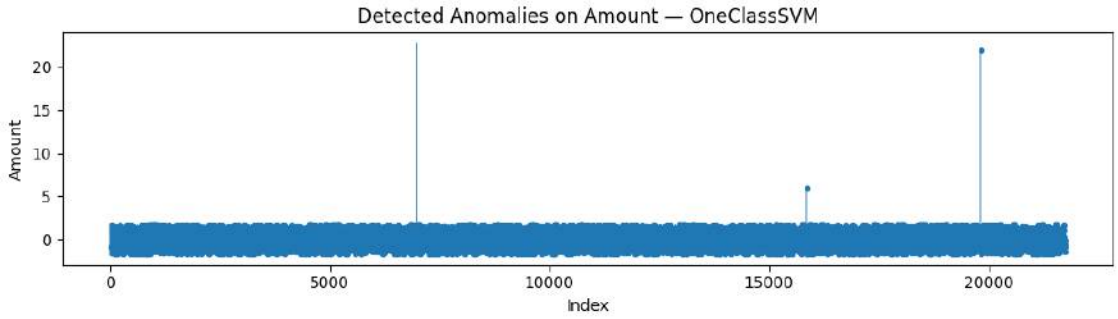


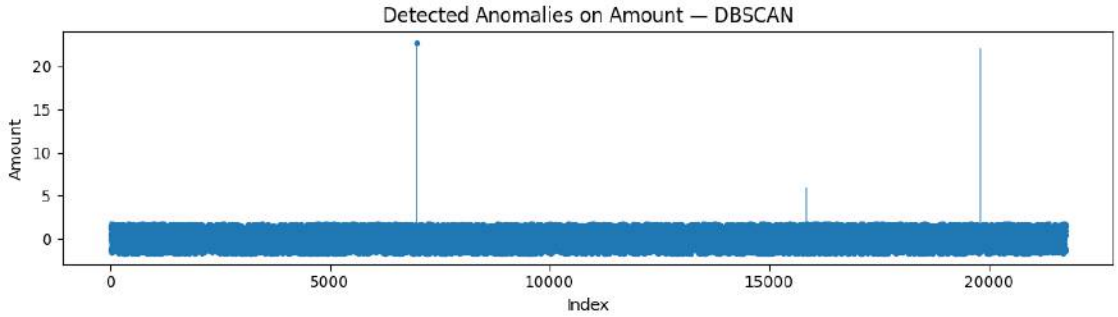

```
<Figure size 800x550 with 0 Axes>
```

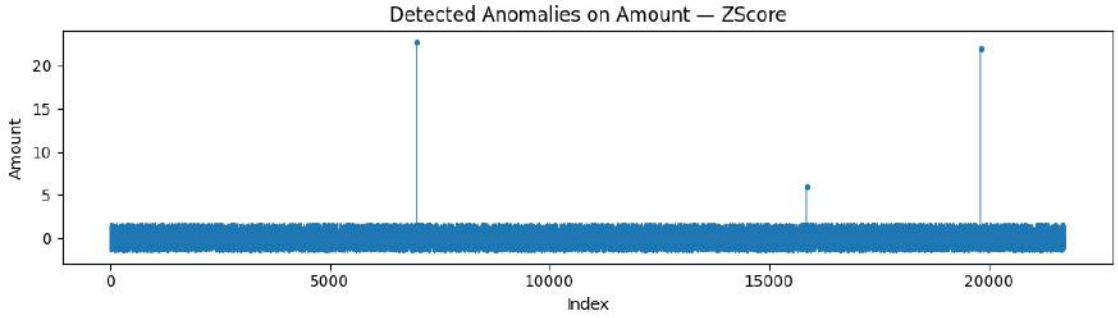

Computational — Train/Score Time (s)
LOF
ZScore
DBSCAN
OneClassSVM
IsolationForest
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Train/Score Time (s)

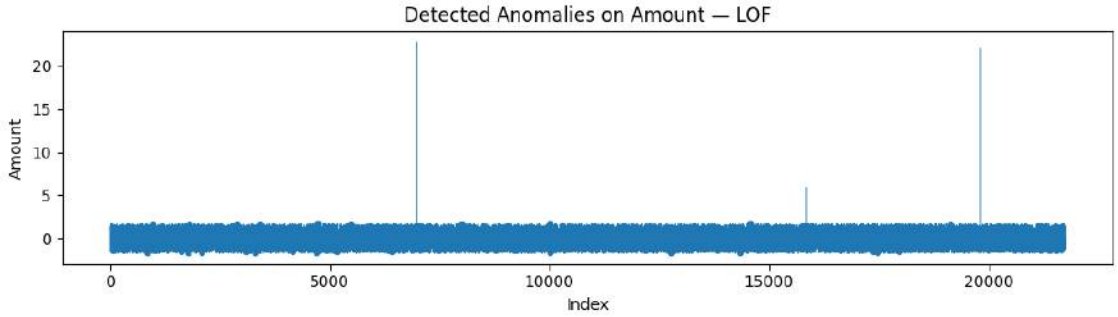

Computational — Δ Memory (MB)
LOF
ZScore
DBSCAN
OneClassSVM
IsolationForest
−600
−400
−200
0
200
400
Δ Memory (MB)


Detected Anomalies on Amount — IsolationForest
Amount
20
15
10
5
0
0
5000
10000
15000
20000
Index


Detected Anomalies on Amount — OneClassSVM
Amount
Index
0
5
10
15
20
0
5000
10000
15000
20000


Detected Anomalies on Amount — DBSCAN
Amount
Index
0
5
10
15
20
0
5000
10000
15000
20000


Detected Anomalies on Amount — ZScore
Amount
Index
0
5
10
15
20
0
5000
10000
15000
20000


Detected Anomalies on Amount — LOF
Amount
Index
0
5
10
15
20
0
5000
10000
15000
20000

0.7 Part 7 — Algorithm Technical Overviews & Comparative Notes

Brief, technical descriptions and method-specific findings.

0.7.1 Isolation Forest

Isolation Forest isolates anomalies by recursively partitioning the data space using random splits; fewer splits are needed to isolate outliers, yielding shorter average path lengths. It performs well on high-dimensional numeric data and scales linearly with sample size.

0.7.2 One-Class SVM

One-Class SVM estimates the support of the normal data distribution by learning a decision boundary in a high-dimensional feature space; points outside the learned boundary are flagged as outliers. Performance depends on the kernel (often RBF) and hyperparameters (`nu`, `gamma`).

0.7.3 DBSCAN

DBSCAN groups densely packed points into clusters based on a neighborhood radius (`eps`) and minimum samples; points not belonging to any dense region (label `-1`) are considered outliers. It handles arbitrary cluster shapes but requires careful `eps` tuning.

0.7.4 Z-Score

Z-score is a statistical method that identifies points with standardized deviations exceeding a threshold (e.g., $|z| > 3$) relative to feature means; it assumes approximately Gaussian distributions and can be sensitive to scale and heavy tails.

0.7.5 Local Outlier Factor (LOF)

LOF measures the local density deviation of a sample relative to its neighbors; points with substantially lower density than neighbors are outliers. It is effective for local density anomalies but depends on the neighborhood size (`n_neighbors`).

0.7.6 What Anomalies Were Detected & How Results Differ

Each method produces a binary anomaly label and a continuous anomaly score.
- **Isolation Forest** and **LOF** tend to highlight sparse, locally isolated points.
- **One-Class SVM** enforces a global boundary and may be stricter or looser depending on `nu`/`gamma`.
- **DBSCAN** flags samples in low-density regions as noise.
- **Z-Score** catches extreme values in a standardized numeric space.

Disagreements between methods often indicate borderline points or regions where density and global-boundary notions diverge.

0.7.7 Choosing a Technique Based on Data Type

- **Structured, mostly numeric, no labels:** Isolation Forest, LOF, or Z-Score (fast baselines). If clusters are expected, DBSCAN helps. One-Class SVM offers tighter boundaries on well-scaled features.
- **High-dimensional sparse:** Isolation Forest scales well; One-Class SVM may be expensive.
- **With labels (supervised/weak labels):** Evaluate via Precision/Recall/F1, ROC-AUC/PR-AUC; calibrate thresholds via business costs.
- **Unstructured (text/images):** Use representation learning first (embeddings/autoencoders), then apply anomaly scorers in the embedding space.

0.7.8 Simple Checks for Outliers

- Boxplots, histograms, and quantiles (e.g., 0.5%, 99.5%)
- Z-Score or IQR rules on key numeric features (e.g., `Amount`)
- Per-account baselines (compare a transaction to that account's historical distribution)
- Time-based spikes (sudden bursts by hour/day), unusual geo/merchant combinations

0.7.9 When to Use Statistical vs. Machine Learning Methods

- **Statistical (Z-Score/IQR/Mahalanobis):** Good for quick baselines, interpretable thresholds, approximately Gaussian features.
- **Machine Learning (Isolation Forest, LOF, One-Class SVM, DBSCAN):** Useful for complex, non-Gaussian patterns and scalable detection in large datasets.

Combine both: use statistical screens to narrow the search, then machine learning for refined detection and ranking.

0.8 Part 8 — Comparative Interpretation & Study Narrative

This study analyzed financial-transaction data to evaluate the behavior and comparative performance of multiple **unsupervised anomaly-detection algorithms**.
The workflow consisted of:
(1) loading and profiling the `financial_anomaly_data.csv` dataset,
(2) performing a comprehensive EDA with histograms, boxplots, correlation checks, and missing-value visualization,
(3) executing systematic preprocessing including imputation for numeric, categorical, and datetime fields; one-hot encoding of categoricals; extraction of temporal features (hour, day-of-week, month); and feature standardization.

A **10 % representative sample** (21 744 rows) was taken for efficiency, and the dataset was retained fully in memory (via `X_out` and `data = data_sampled`) to avoid repeated I/O. Because

only **0.22 % of all cells** contained missing values, these were imputed rather than dropped to preserve data completeness and model comparability.

Five algorithms were applied: **Isolation Forest**, **One-Class SVM**, **DBSCAN**, **Z-Score**, and **Local Outlier Factor (LOF)**. Each produced both binary anomaly flags and continuous anomaly-score distributions. Where ground-truth labels were absent—as in this dataset— evaluation emphasized **unsupervised metrics** (score distributions, statistical summaries), **computational efficiency** (time and Δ memory), and **qualitative visual inspection** of detected anomalies on the *Amount* feature.

Key comparative insights

- **Isolation Forest** remained the most efficient, scaling linearly with sample size and providing stable anomaly rankings.
- **One-Class SVM** offered tighter global boundaries but was computationally heavier, consistent with kernel-based complexity.
- **DBSCAN** effectively flagged low-density transaction regions yet required tuning of `eps` and `min_samples`.
- **Z-Score** served as a transparent statistical baseline, quickly identifying extreme monetary values.
- **LOF** captured local-density irregularities and complemented the global sensitivity of Isolation Forest.

Because no labeled anomalies were available, **performance, ranking-based, and domain metrics** (e.g., Precision, Recall, F1, ROC-AUC, Precision@K) were intentionally omitted—these require known ground truth. Instead, **statistical score summaries** and **computational trade-offs** guided interpretation.

Overall conclusion

For structured, predominantly numeric financial data with minimal missingness and no labels, the combination of **Isolation Forest + LOF** provides a robust, interpretable baseline. **DBSCAN** adds value when transaction clusters and noise are expected, while **Z-Score** remains a simple benchmark for transparency.
Future work may extend this analysis by incorporating temporal-sequence modeling or partial fraud labels to enable supervised and ranking-based evaluation frameworks.

[ ]: